

KD_vWS

October 31, 2025

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

1 Part II: vWS Development

This notebook presents the mathematical details, plots, and descriptions involved in developing the virtual Win-Shares metric vWS.

1.1 Summary:

After carrying out and implementing everything below to build vWS, I achieve a significant improvement over the traditional WS metric.

Correlation between new vWS and Team Quality: 0.8748

Correlation between traditional WS and Team Quality: 0.7757

This gives confidence in proceeding to build the player value app based on vWS.

1.2 Merging necessary data frame to perform analysis.

Read in scraped data from College Basketball Reference (player stats) and Kenpom.com (team stats) and merge together.

```
[3]: cbb=pd.read_csv('cbb_full_data.csv')
del cbb['Unnamed: 0']
```

Now merge the player stats data with team outcome data from Ken Pomeroy (<https://kenpom.com/>). Since school naming conventions vary between sites, use the `cbb_kenpom_team_names_lookup_table` file to facilitate merges.

```
[4]: kenpom=pd.read_csv('KenPomDataSince2004.csv')
```

First change annoying date structure that somehow got in there – 20-Dec should be a W-L of 12-20. Probably happened from passing through Excel with the kenpom data.

```
[5]: def fixwl(row):
    ↵
    ↪molist=["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"]
```

```

modict={"Jan":'1',"Feb":'2',"Mar":'3',"Apr":'4',"May":'5',"Jun":'6',"Jul":
↪'7',"Aug":'8',"Sep":'9',"Oct":'10',"Nov":'11',"Dec":'12'}
if row['W-L'].split("-")[1] in molist:
    x=row['W-L'].split("-")[1]
    y=row['W-L'].split("-")[0]
    return modict[x]+"-"+y
else:
    return row['W-L']

```

```
[6]: kenpom["W-L"]=kenpom.apply(fixwl,axis=1)
```

Split W-L into separate W and L columns.

```
[7]: def wins(row):
      return int(row['W-L'].split("-")[0])
def losses(row):
      return int(row['W-L'].split("-")[1])

```

```
[8]: kenpom['Wins']=kenpom.apply(wins,axis=1)
kenpom['Losses']=kenpom.apply(losses,axis=1)
```

Import team name lookup table

```
[9]: tm_name_lookup=pd.read_csv('cbb_kenpom_team_names_lookup_table.csv')
```

Merge together player stats and team info:

```
[10]: kenpom=kenpom.merge(tm_name_lookup,how='left',left_on='TeamName',right_on='KP_
↪Team Name')
```

```
[11]: kp11on=kenpom[kenpom['Season']>=2011].reset_index(drop=1)
```

```
[12]: kp11on.to_csv('kenpom_2011_on',index=0)
```

```
[13]: kp11on_lim=kp11on[['Season', 'Tempo', 'RankTempo', 'AdjTempo', 'RankAdjTempo',
      'OE', 'RankOE', 'AdjOE', 'RankAdjOE', 'DE', 'RankDE', 'AdjDE',
      'RankAdjDE', 'AdjEM', 'RankAdjEM', 'Rk', 'W-L', 'AdjO', 'AdjD',
      'AdjOppOE', 'AdjOppDE', 'Wins', 'Losses', 'CBB Team Link']]

```

```
[14]: cbb_kp=cbb.
      ↪merge(kp11on_lim,how='left',left_on=['Season','Team'],right_on=['Season','CBB_
      ↪Team Link'])

```

Check for missing data after merge – Looks good.

```
[15]: cbb_kp.loc[cbb_kp['AdjEM'].isna()][['Team','Season']]['Team'].unique()
```

```
[15]: array([], dtype=object)
```

```
[16]: cbb_kp.loc[cbb_kp['CBB Team Link'].isna()]['Team'].unique()
```

```
[16]: array([], dtype=object)
```

Add individual player possession column IndOffPoss. This is required later in the WS formula.

For each player, use

$$ORtg = 100 * (PointsProduced) / (IndOffPoss)$$

to estimate IndOffPoss.

Also, we fix USG% for cases with extreme outlier values due to very low minutes played.

```
[17]: def indposs(row):  
    if ((row['PProd']==0)&(row['MP']>0)):  
        return 0  
    elif ((row['ORtg']>0)&(row['MP']>0)):  
        return row['PProd']*100/row['ORtg']  
    else:  
        return np.nan  
  
def repair_usg(row):  
    if (row['USG%']>=40)&(row['MP']<=100):  
        return 17.15  
    else:  
        return(row['USG%'])
```

```
[18]: cbb_kp['IndOffPoss']=cbb_kp.apply(indposs,axis=1)  
cbb_kp['USG%']=cbb_kp.apply(repair_usg,axis=1)
```

1.3 Analyzing and addressing the relationship between Efficiency and Usage

WS is calculated in two parts as OWS (Offensive Win Shares) + DWS (Defensive Win Shares).

WS formula: <https://www.sports-reference.com/cbb/about/ws.html>

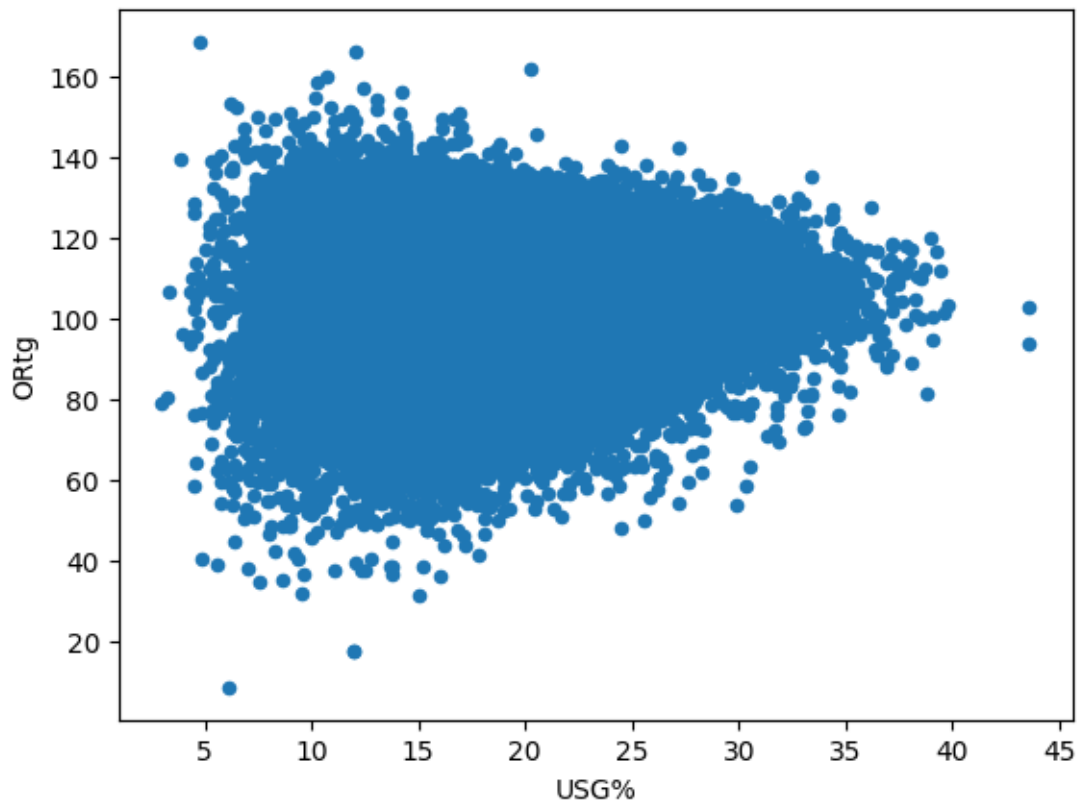
I focus on OWS and make only one small adjustment to DWS later. The main ingredient in OWS is Offensive Rating (ORtg = player points per 100 individual possessions). That's the main focus of this analysis.

Here's the main issue we're trying to deal with.

Offensive Rating (ORtg) increases slightly as Usage% (USG%: %of team possessions used by a player) increases, but variance is much larger at lower values of USG%. This means that the most and least efficient players *always* occur at lower values of USG%. Especially in college basketball, where there is a premium on players who can create their own shots/offense and tend to have higher usage, this fails to properly reflect value.

The cone-shape in the scatterplot below shows this relationship clearly. Lower USG% has higher variance; higher USG% has lower variance.

```
[19]: cbb_kp[cbb_kp['MP']>=100].plot.scatter(x='USG%',y='ORtg')
plt.show()
```



At the same time there *are* lower USG% players who are highly efficient and have high value. Ultimately we want to properly reflect the value of higher USG% players while still allowing some high efficiency players to appear as high value at lower USG%.

In other sports like baseball, Bayesian methods have been used to make comparisons between players with widely varying numbers of at-bats, where a similar phenomenon exists. (See, for example: <http://varianceexplained.org/r/simulation-bayes-baseball/>)

In that case, using a beta-distribution fit to the data they can effectively pull low at-bat players toward the mean while, for higher at-bat players, the actual values of hits and at-bats dominate the beta parameters (so the high at-bat players are “shrunk” less).

The empirical approach here used a weighted average of the point’s ORtg and the predicted ORtg on the regression line.

Unfortunately, I could not get this to work well with my data. It reversed the shape of the cone completely, but shrunk lower USG% players towards the regression line too much. Lower USG% players had very little ability to “stand-out”. Other attempts to moderate the shrinkage in related ways similarly failed to give reasonable results.

1.4 Solution

Adjust Offensive Rating based on Usage as follows:

Building on the ideas above but taking a different (simpler?) approach:

1. Fit the regression line giving Offensive Rating as a function of Usage %. So at USG% u , $OR\hat{t}g_u = b_0 + b_1 \cdot (u)$
2. Compute the residuals. For point i , at USG% u , $r_{i,u} = ORtg_{i,u} - OR\hat{t}g_u$
3. Fit another regression to the squared residuals as a function of USG%. At USG% u we have $\hat{r}_{i,u}^2 = c_0 + c_1 \cdot (u)$.

Let s_u be the estimated standard deviation (using the regression in 3) of ORtg at the USG% value u .

Let $s_{\bar{u}}$ be the estimated standard deviation of ORtg at the average value $\bar{u}$ of USG%.

Note: From the data the average value of USG% is 19.

4. For each observation, calculate the z -score of its ORtg based on estimated standard deviation at the observation's USG% value u .

$$z_{i,u} = \frac{ORtg_{i,u} - OR\hat{t}g_u}{s_u}$$

So $z_{i,u}$ is the number of s_u -standard deviations above (below) the regression line observation i is at USG% u .

5. For observation i with USG% u , replace $ORtg_{i,u}$ with $OR\hat{t}g_u + z_{i,u} \cdot s_{\bar{u}}$.

So, if a player with USG% u has an ORtg that is 2 s_u -standard deviations above (below) the predicted value on the regression line, we would move him to be 2 $s_{\bar{u}}$ -standard deviations above (below) the value on the line.

Since high usage players have smaller than average s_u -standard deviations ($s_u < s_{\bar{u}}$), this has the effect of “stretching” them further from the line. For lower usage players, $s_u > s_{\bar{u}}$ so this has the effect of shrinking them towards the line. The effect is to create a relationship between adjusted ORtg and USG that looks more reasonable (see below) by creating more constant variance around the regression line.

6. The scatterplot (below) looks better but *still* seems to overvalue low USG%. A lot less than before though. We can make one more adjustment by penalizing a player further for every percentage point below average USG%.
7. Multiply by a strength of schedule factor (Team Points per Possession / Opponents Points per Possession) to get adjusted Offensive Rating: adjORTg.
8. Use adjORTg in place of ORtg in the WS formula. See <https://www.sports-reference.com/cbb/about/ws.html> for an overview of the formula. Win Shares is computed in two parts: Offensive (OWS) and Defensive (DWS) Win shares. I also added a strength of schedule weight to defensive win shares but otherwise left it alone.

This is all implemented here with some minor adjustments to ignore low-minute or otherwise extreme outliers.

1.5 Implementation

```
[20]: #import for regressions
import statsmodels.formula.api as sm
```

First build the $ORTg \sim USG\%$ model.

```
[21]: ORtg_USG_Mod=sm.ols(formula="ORTg ~ Q('USG%')", data=cbb_kp[cbb_kp['MP']>=100]).
    ↪fit()
```

Compute residuals.

```
[22]: cbb_kp['res'] = cbb_kp['ORTg']-ORTg_USG_Mod.predict(cbb_kp['USG%'])
```

Truncate extreme residuals so they don't skew the models:

```
[23]: def trunc_res(row):
    if (row['res']>40):
        return 40
    elif (row['res']<=(-40)):
        return -40
    else:
        return(row['res'])
```

```
[24]: cbb_kp['trunc_res']=cbb_kp.apply(trunc_res,axis=1)
```

Compute squared residuals after truncating.

```
[25]: cbb_kp['trunc_res2']=cbb_kp['trunc_res']**2
```

Build squared residual model. Exclude more extreme values in model build:

```
[26]: res2_mod=sm.ols(formula = "trunc_res2 ~ Q('USG%')",
    ↪data=cbb_kp[(cbb_kp['MP']>=100)&(cbb_kp['trunc_res2']<1600)]).fit()
```

Define the z -score for each point using the appropriate value of standard deviation based on the point's $USG\%$ value:

```
[27]: def sd2_adj(row):
    sd = np.sqrt(max((res2_mod.params['Intercept']+res2_mod.
    ↪params["Q('USG%')"]*row['USG%']), (res2_mod.params['Intercept']+res2_mod.
    ↪params["Q('USG%')"]*35)))
    yhat = ORtg_USG_Mod.params['Intercept']+ORTg_USG_Mod.
    ↪params["Q('USG%')"]*row['USG%']
    return (row['ORTg']-yhat)/sd
```

Create a column giving the z -score at u for each observation.

```
[28]: cbb_kp['sd2']=cbb_kp.apply(sd2_adj,axis=1)
```

Find the standard deviation at the average usage value of 19.

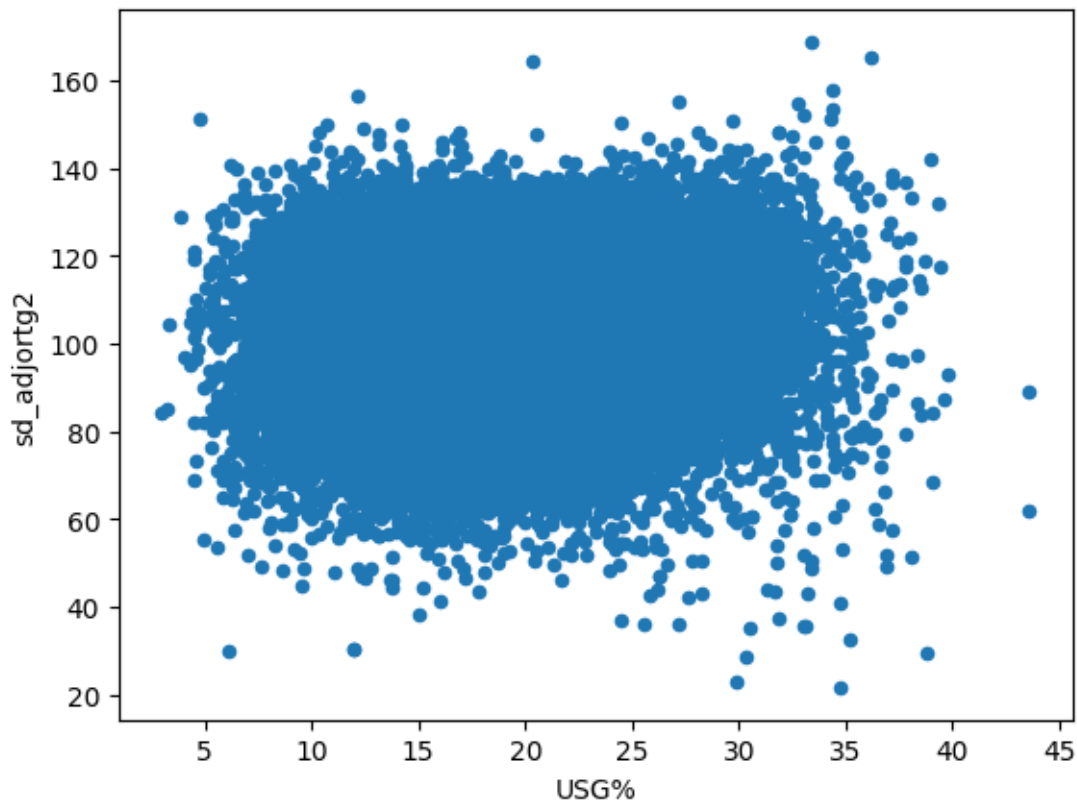
```
[29]: sd2_avg=np.sqrt(res2_mod.params['Intercept']+res2_mod.params["Q('USG%')"]*19)
```

Now adjust ORtg as defined above, but without the strength of schedule adjustment or the low-USG% penalty. This is kind of a temporary adjusted ORtg but not the final one.

```
[30]: def sdadjortg(row):  
    yhat = ORtg_USG_Mod.params['Intercept']+ORtg_USG_Mod.  
    ↪params["Q('USG%')"]*row['USG%']  
    return (yhat + row['sd2']*sd2_avg)
```

```
[31]: cbb_kp['sd_adjortg2']=cbb_kp.apply(sdadjortg,axis=1)
```

```
[32]: cbb_kp[cbb_kp['MP']>=100].plot.scatter(x='USG%',y='sd_adjortg2')  
plt.show()
```



You can see that this looks better but it still bends up a little for low USG%. The low-usage penalty below will try to pull those down a little in the final adjustment.

To implement the strength-of schedule weight I need the average team efficiency for each season. This is stored in the dictionary below. The avgPPGdict below is the dictionary of average points

per game for each season in college basketball. This is needed in the Win Shares formula used below.

Note: Strength of schedule adjustment is just the average points per 100 possessions in college basketball divided by a team's opponents average points allowed per 100 possessions. You get multiplied by a factor greater than 1 if you played teams who on average allowed less than the average points.

```
[33]: adj0dict=dict(  
    sn2011=103.23,  
    sn2012=102.81,  
    sn2013=102.38,  
    sn2014=105.0,  
    sn2015=104.16,  
    sn2016=104.72,  
    sn2017=104.51,  
    sn2018=104.82,  
    sn2019=104.09,  
    sn2020=102.13,  
    sn2021=102.35,  
    sn2022=102.77,  
    sn2023=104.89,  
    sn2024=105.99,  
    sn2025=106.60  
)  
  
    avgPPGdict=dict(  
    sn2011=68.4,  
    sn2012=67.4,  
    sn2013=66.9,  
    sn2014=70.2,  
    sn2015=66.9,  
    sn2016=72.3,  
    sn2017=72.7,  
    sn2018=73.6,  
    sn2019=71.9,  
    sn2020=70.2,  
    sn2021=70.8,  
    sn2022=70.1,  
    sn2023=70.7,  
    sn2024=72.6,  
    sn2025=72.8)
```

```
[34]: def sos(row):  
    season = 'sn'+str(row['Season'])  
    return adj0dict[season]/row['AdjOppDE']
```

```
[35]: cbb_kp['sos_factor']=cbb_kp.apply(sos,axis=1)
```

Add a small adjustment to defensive Win Shares using a similar strength of schedule adjustment as above.

```
[36]: def adjustDWS(row):  
       return row['DWS']*row['AdjOppOE']/103
```

```
[37]: cbb_kp['vDWS']=cbb_kp.apply(adjustDWS,axis=1)
```

Add in a low-USG% penalty and SOS weight.

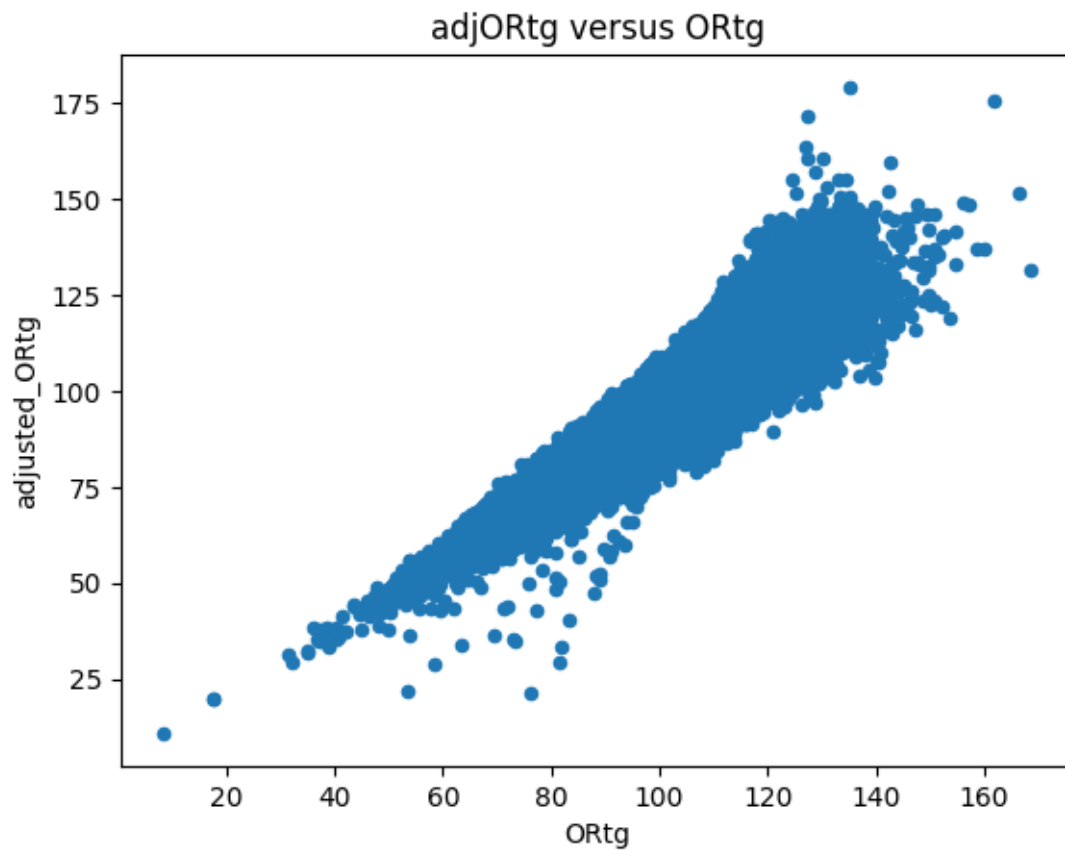
This is final adjORTg which is the key ingredient in the WS formula.

```
[38]: def adjust2(row):  
       season = 'sn'+str(row['Season'])  
       if (row['USG%']<19):  
           return (row['sd_adjortg2']+1.5*(row['USG%']-19))*(adj0dict[season])/  
           ↪row['AdjOppDE']  
       else:  
           return row['sd_adjortg2']*(adj0dict[season])/row['AdjOppDE']
```

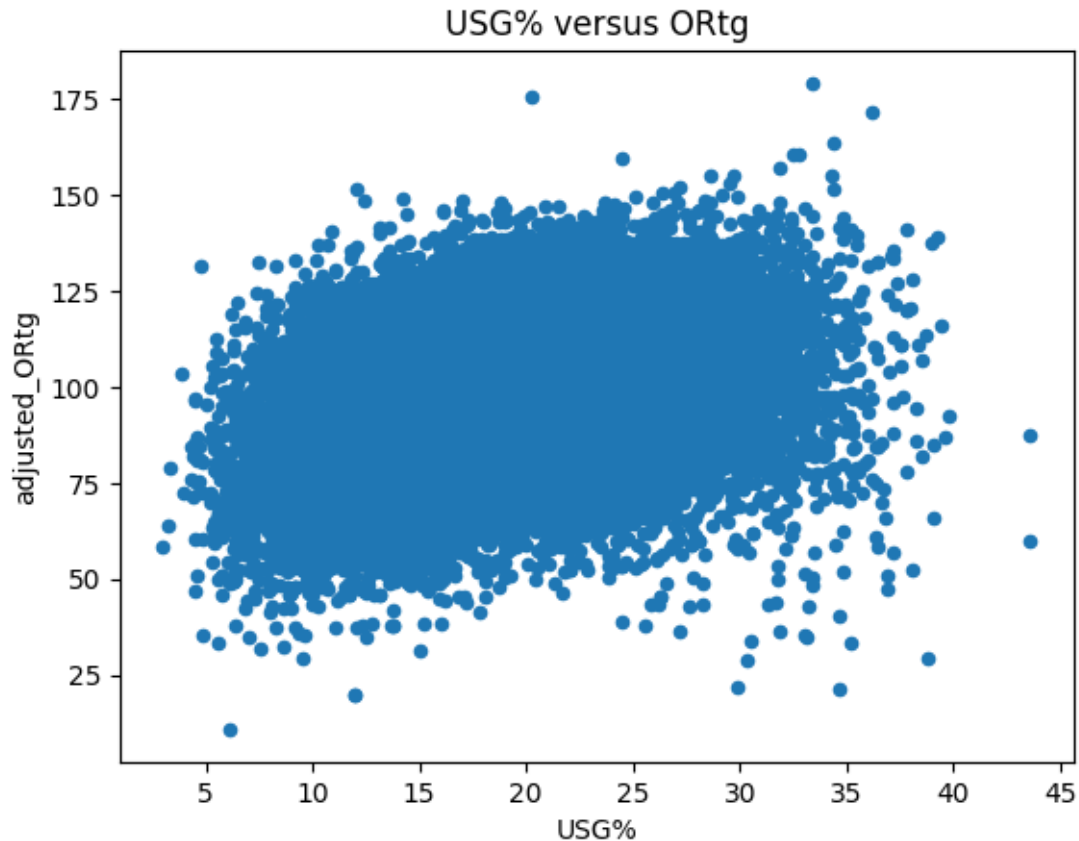
```
[39]: cbb_kp['adjusted_ORtg']=cbb_kp.apply(adjust2,axis=1)
```

Let's check adjORTg compared to both ORtg and USG%.

```
[40]: cbb_kp[cbb_kp['MP']>=100].plot.scatter(x='ORtg',y='adjusted_ORtg')  
       plt.title('adjORTg versus ORtg')  
       plt.show()
```



```
[41]: cbb_kp[cbb_kp['MP']>=100].plot.scatter(x='USG%',y='adjusted_ORTg')
plt.title('USG% versus ORtg')
plt.show()
```



Note that

1. The cone shape is gone.
2. The variance is more constant.
3. The upward trend is preserved.

Finally define vOWS following the OWS formula but using adjORtg as input. (See <https://www.sports-reference.com/cbb/about/ws.html> for details on the Win Shares formula.)

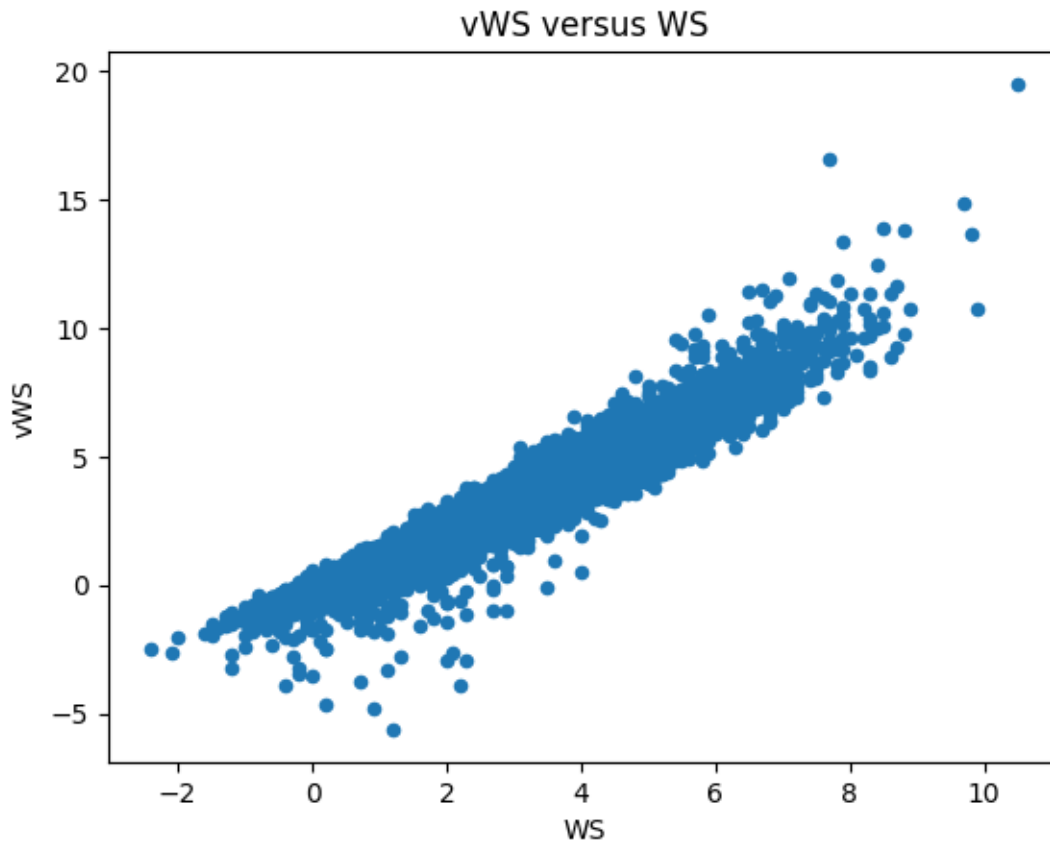
```
[42]: def vOWS(row):
      season = 'sn'+str(row['Season'])
      return (((row['adjusted_ORtg']/100)-.875*1.03)*row['IndOffPoss'])/((
      ↪5*avgPPGdict[season])*row['AdjTempo']/66.8)
```

```
[43]: cbb_kp['vOWS']=cbb_kp.apply(vOWS,axis=1)
```

```
[44]: cbb_kp['vWS']=cbb_kp['vOWS']+cbb_kp['vDWS']
```

Check vWS versus traditional WS.

```
[45]: cbb_kp[cbb_kp['MP']>=100].plot.scatter(x='WS',y='vWS')
plt.title('vWS versus WS')
plt.show()
```



They are very correlated which makes sense but there are some changes.

2 Did it work?

Ken Pomeroy's Adjusted Efficiency Margin (adjEM) is the standard metric to evaluate team quality. Every team uses it and it's part of the NCAA Tournament evaluation process. Let's check correlation between adjEM and traditional WS and vWS summed over each team.

```
[46]: vws_test=cbb_kp[cbb_kp['MP']>=100][['WS','vWS','Team','Season']].
      ↳groupby(['Team','Season'],as_index=0).sum()
adjem=cbb_kp[['Team','Season','AdjEM']].groupby(['Team','Season'],as_index=0).
      ↳mean()
```

```
[47]: vws_test2=vws_test.merge(adjem,how='left',on=['Team','Season'])
```

```
[48]: vws_test2[['WS','vWS','AdjEM']].corr().round(4)
```

```
[48]:      WS      vWS   AdjEM
WS      1.0000  0.9531  0.7757
vWS      0.9531  1.0000  0.8748
AdjEM    0.7757  0.8748  1.0000
```

3 Conclusion

Yes! *There's a significant jump in correlation between Team Quality (AdjEM) and my vWS compared to traditional WS.*

vWS also looks good in that several of the all time college greats are at the top of the list:

```
[49]: cbb_kp[cbb_kp['MP']>=100][['Player', 'Team', 'W-L', 'Season', 'MP', 'USG%', 'vWS', 'WS']].
      ↪sort_values(by='vWS', ascending=0)[0:25]
```

```
[49]:      Player      Team  W-L  Season   MP  \
826      Zach Edey      purdue   34-5    2024  1248.0
43726     Doug McDermott    creighton   27-8    2014  1181.0
56342     Kemba Walker    connecticut   32-9    2011  1543.0
62575     Jimmer Fredette  brigham-young   32-5    2011  1323.0
13094     Zach Edey      purdue   29-6    2023  1077.0
14548     Frank Kaminsky  wisconsin   36-4    2015  1311.0
54382     Doug McDermott    creighton   28-8    2013  1138.0
32944     Cooper Flagg     duke   35-3    2025  1135.0
66863     Reggie Hamilton    oakland   20-16    2012  1284.0
71510  Aniwanika Tait-Jones  california-san-diego   17-15    2025  1042.0
44899     Keegan Murray      iowa   26-10    2022  1116.0
73855     Mike Daum    south-dakota-state   18-17    2017  1143.0
70099     T.J. Warren  north-carolina-state   22-14    2014  1238.0
20667     Chris Clemons    campbell   20-13    2019  1208.0
4223     Trey Burke     michigan   31-8    2013  1378.0
33082     Zion Williamson     duke   32-6    2019   990.0
14923     Damian Lillard    weber-state   25-7    2012  1105.0
40246     Luka Garza      iowa   22-9    2021   975.0
39580     Buddy Hield    oklahoma   29-8    2016  1309.0
66457     Johni Broome     auburn   32-5    2025  1088.0
223     Cassius Winston    michigan-state   32-7    2019  1308.0
58698     Darius McGhee     liberty   27-9    2023  1150.0
36448     Jaedon Ledee    san-diego-state   26-11    2024  1174.0
70042     Shabazz Napier    connecticut   32-8    2014  1404.0
60198     Ja Morant     murray-state   28-5    2019  1207.0

      USG%      vWS   WS
826    33.4  19.495031  10.5
43726   36.2  16.558108   7.7
56342   32.4  14.879473   9.7
62575   37.8  13.868230   8.5
```

13094	32.8	13.807333	8.8
14548	28.6	13.628445	9.8
54382	34.3	13.370607	7.9
32944	30.9	12.474454	8.4
66863	34.8	11.967586	7.1
71510	29.5	11.880377	7.8
44899	29.7	11.655150	8.7
73855	34.4	11.475421	6.7
70099	35.5	11.445867	6.5
20667	39.0	11.367219	7.5
4223	28.3	11.361625	8.6
33082	28.6	11.343764	8.3
14923	33.0	11.336884	8.0
40246	32.5	11.282210	6.9
39580	30.2	11.194428	7.6
66457	30.4	11.013952	6.8
223	28.6	11.008442	7.7
58698	35.5	10.993383	7.4
36448	30.5	10.892519	7.4
70042	27.5	10.780025	7.9
60198	33.3	10.752556	8.2

```
[50]: #cbb_kp.to_csv('cbb_kp_with_vWS_October.csv',index=0)
```

Final Note: Recall that WS had the interpretation of adding up (over players) to about the number of team wins. That is still basically true. Mean absolute error over all teams is 2.0 for WS, and for vWS is 3.0.

```
[ ]:
```